ChatGPT

손글씨 데이터 텐서플로우 모델 제작, 분류리포트, 시각화

손글씨 데이터로 텐서플로우 모델을 제작하고, 분류 성능을 리포트하고, 결과를 시각화하는 과정은 크게 3단계로 나눌 수 있습니다: **데이터 준비**, **모델 훈련 및 평가**, **결과 시각화**입니다. 여기서는 대표적인 손글씨 데이터셋인 **MNIST**를 사용하여 이를 구현하는 방법을 소개하겠습니다.

1. 데이터 준비

손글씨 데이터셋인 MNIST는 TensorFlow와 Keras에서 이미 내장되어 있어 쉽게 불러와 사용할 수 있습니다. 이 데이터셋은 28x28 크기의 그레이스케일 이미지를 포함하며, 각 이미지에는 0부터 9까지의 숫자가 적혀 있습니다.

2. 텐서플로우 모델 훈련

필요 라이브러리 설치 및 불러오기:

```python
import tensorflow as tf
from tensorflow.keras import layers, models
from sklearn.metrics import classification_report
import matplotlib.pyplot as plt
```

MNIST 데이터셋 로딩:

```python
# MNIST 데이터셋 로딩
(train_images, train_labels), (test_images, test_labels) = tf.keras.datasets.mnist.load_data
()

# 데이터를 0과 1 사이로 정규화
train_images = train_images / 255.0
test_images = test_images / 255.0

# 데이터 차원 변환 (28, 28, 1)
train_images = train_images.reshape((train_images.shape[0], 28, 28, 1))
test_images = test_images.reshape((test_images.shape[0], 28, 28, 1))
```

모델 정의:

```python
# 모델 정의
model = models.Sequential([
    layers.Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.MaxPooling2D((2, 2)),
    layers.Conv2D(64, (3, 3), activation='relu'),
    layers.Flatten(),
    layers.Dense(64, activation='relu'),
    layers.Dense(10, activation='softmax')
])

# 모델 컴파일
model.compile(optimizer='adam',
              loss='sparse_categorical_crossentropy',
              metrics=['accuracy'])
```

모델 훈련:

```python
history = model.fit(train_images, train_labels, epochs=5, validation_data=(test_images, test
_labels))
```

3. 성능 평가 및 리포트

모델 평가 (테스트 데이터셋으로):

```python
test_loss, test_acc = model.evaluate(test_images, test_labels)
print(f"Test accuracy: {test_acc:.4f}")
```

분류 리포트 생성:

```python
# 예측 결과
test_predictions = model.predict(test_images)
test_pred_labels = test_predictions.argmax(axis=1)

# 분류 리포트
print(classification_report(test_labels, test_pred_labels))
```

분류 리포트 예시 (출력):

```plaintext
              precision    recall  f1-score   support

           0       0.98      0.99      0.98       980
```

```
              1       0.99      0.99      0.99      1135
              2       0.98      0.97      0.98      1032
              3       0.97      0.97      0.97      1010
              4       0.97      0.97      0.97       982
              5       0.96      0.97      0.97       892
              6       0.98      0.98      0.98       958
              7       0.97      0.98      0.97      1028
              8       0.98      0.96      0.97       974
              9       0.97      0.97      0.97      1009

       accuracy                           0.98     10000
      macro avg       0.98      0.98      0.98     10000
   weighted avg       0.98      0.98      0.98     10000
```

4. 결과 시각화

훈련 및 검증 정확도, 손실을 시각화하여 모델 학습 과정을 분석할 수 있습니다.

```python
# 훈련 및 검증 정확도 시각화
plt.figure(figsize=(12, 6))

# 정확도
plt.subplot(1, 2, 1)
plt.plot(history.history['accuracy'], label='Training accuracy')
plt.plot(history.history['val_accuracy'], label='Validation accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.title('Training and Validation Accuracy')
plt.legend()

# 손실
plt.subplot(1, 2, 2)
plt.plot(history.history['loss'], label='Training loss')
plt.plot(history.history['val_loss'], label='Validation loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.title('Training and Validation Loss')
plt.legend()

plt.show()
```

5. 테스트 이미지 예측 결과 시각화

```python
# 테스트 이미지 중 몇 개를 예측하고 시각화
num_images = 5
plt.figure(figsize=(12, 4))

for i in range(num_images):
    plt.subplot(1, num_images, i + 1)
    plt.imshow(test_images[i].reshape(28, 28), cmap=plt.cm.binary)
    plt.title(f"Pred: {test_pred_labels[i]}")
    plt.axis('off')

plt.show()
```

결론

위의 코드를 통해 손글씨 데이터셋(MNIST)을 사용하여 텐서플로우 모델을 구축하고, 성능을 평가한 뒤, 분류 리포트와 훈련 과정, 예측 결과를 시각화할 수 있습니다. 모델의 정확도와 손실을 그래프로 나타내어 학습 과정을 추적하고, 예측된 손글씨 숫자를 시각화하여 결과를 확인할 수 있습니다.

모델 저장 새로운 데이터로 추론

모델을 저장하고 새로운 데이터로 추론하는 과정을 단계별로 설명드리겠습니다. 텐서플로우에서는 모델을 **저장**하고, **불러와서 추론**하는 방법을 제공합니다. 이를 통해 훈련된 모델을 나중에 재사용하거나 새로운 데이터에 대해 예측을 수행할 수 있습니다.

1. 모델 저장

훈련된 모델을 **파일로 저장**하는 방법은 `model.save()` 함수를 사용하는 것입니다. 이렇게 저장한 모델은 나중에 불러와서 추론을 수행할 수 있습니다.

모델 저장:

```python
# 모델을 HDF5 형식으로 저장
model.save('my_mnist_model.h5')

# 또는 SavedModel 형식으로 저장 (TensorFlow의 기본 형식)
# model.save('my_mnist_model')
```

- **HDF5 형식** (`.h5` 파일): 텐서플로우/Keras 모델을 저장하는 전통적인 방식으로, 모델 구조와 가중치, 옵티마이저 상태 등을 포함합니다.
- **SavedModel 형식**: 텐서플로우에서 권장하는 모델 저장 형식으로, `SavedModel` 폴더에 모델 구조와 가중치, 메타데이터가 포함됩니다.

2. 모델 불러오기

저장된 모델을 불러오는 방법은 `tf.keras.models.load_model()`을 사용하는 것입니다.

모델 불러오기:

```python
# HDF5 형식으로 저장한 모델 불러오기
loaded_model = tf.keras.models.load_model('my_mnist_model.h5')

# SavedModel 형식으로 저장한 모델 불러오기 (폴더 경로를 지정)
# loaded_model = tf.keras.models.load_model('my_mnist_model')
```

3. 새로운 데이터로 추론 (예측)

모델을 불러온 후, 새로운 데이터로 추론을 수행할 수 있습니다. 예를 들어, 새로운 손글씨 이미지를 모델에 입력하여 예측을 받을 수 있습니다.

새로운 데이터 준비:

새로운 데이터를 테스트하기 전에, 모델이 훈련된 데이터와 동일한 형식으로 데이터를 준비해야 합니다. 예를 들어, MNIST 데이터와 같은 28x28 크기의 흑백 이미지일 경우, 이를 `0~1` 범위로 정규화하고, 적절한 형태로 변환해야 합니다.

```python
import numpy as np
from tensorflow.keras.preprocessing import image

# 새로운 이미지 로드 (예: 로컬 파일에서)
img_path = 'path_to_your_image.png'

# 이미지를 28x28 크기로 리사이즈하고, 그레이스케일로 변환
img = image.load_img(img_path, target_size=(28, 28), color_mode='grayscale')

# 이미지를 numpy 배열로 변환하고, 정규화
img_array = np.array(img) / 255.0

# 모델 입력 형태로 변환 (배치 차원 추가)
img_array = img_array.reshape((1, 28, 28, 1))

# 예측 수행
predictions = loaded_model.predict(img_array)

# 예측 결과 (가장 높은 확률을 가진 클래스)
predicted_class = np.argmax(predictions, axis=1)

print(f'Predicted class: {predicted_class[0]}')
```

4. 예측 결과 시각화

새로운 이미지를 시각화하여, 모델의 예측과 함께 보여줄 수 있습니다.

```python
import matplotlib.pyplot as plt

# 이미지를 시각화
plt.imshow(img_array.reshape(28, 28), cmap=plt.cm.binary)
plt.title(f"Predicted: {predicted_class[0]}")
plt.axis('off')
plt.show()
```

전체 흐름 요약

1. **모델 저장**: `model.save()`를 사용하여 훈련된 모델을 저장.
2. **모델 불러오기**: `tf.keras.models.load_model()`을 사용하여 저장된 모델을 불러옴.
3. **새로운 데이터로 추론**:
   - 이미지를 로드하고, 적절히 전처리한 후 모델에 입력.
   - 모델을 사용하여 예측을 수행.
4. **예측 결과 시각화**: 예측된 결과를 출력하고, 이미지를 시각화.

이 과정을 통해 훈련된 모델을 저장하고, 새로운 데이터를 입력받아 추론할 수 있습니다.